Plain S_5 C_4 with the D_4 oracle's solver options

Question. `src/cryptanalysis/jacobian_transfer/cost.rs` charged every `k = 4` extension index-calculus decomposition $C_4 = 1.227 \cdot 10^{12}$ `F_p` multiplications: §11.17's plain S_5 solve (`research/notes/index-calculus/RESEARCH_RESIDUAL_WALKS.md`), measured before #1792 added sparsest pivots, the chain eigen step and shape replay to the same Macaulay solver for the D_4-symmetric oracle (where they cut C_4 about 370×, 26× of it from sparsest pivots). Do those options, which do not depend on the D_4 form, also cut the plain S_5 solve?

Answer: yes, 13.9×. On §11.17's own instances the plain solve costs $C_4 = 8.69 \cdot 10^{10} \approx 2^{36.34}$ with all three options (mean over 12 replayed decompositions, the one replay fallback charged), against $1.209 \cdot 10^{12}$ for the committed solver on the same residuals. `cost.rs` now charges $8.69 \cdot 10^{10}$. No route class and no curve verdict in `docs/curves/jacobian-transfers.json` changes: the registry has no `k = 4` extension index-calculus row.

Class (AGENTS.md "Success criterion"): an **engineering** improvement of a stage (C_4, one decomposition), measured on counted `F_p` multiplications. It is not an end-to-end `S`; no `k = 4` plain pipeline exists to move.

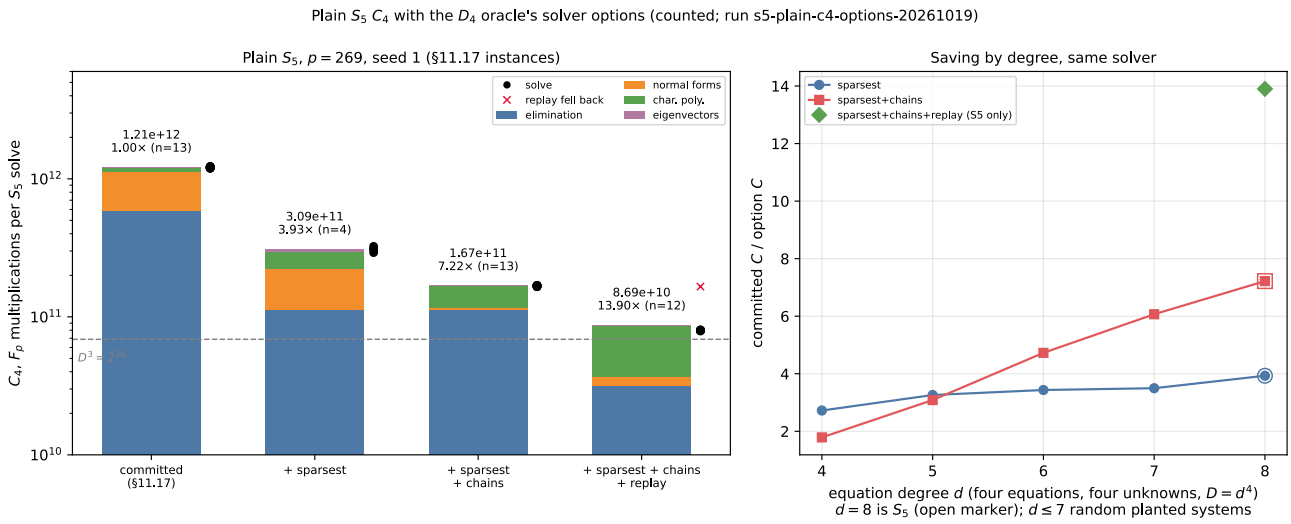


Figure 1: C4 by option

Setup

- **Solver.** `gaudry_quartic::solve_system_w_inner` with weights `(1,1,1,1)` (the plain system has no $\sigma_4 = \pi^2$ to substitute, so #1792's `weighted` option does not apply). The new public wrapper `solve_s5_subspace_with` / `solve_system4_with` (`PlainOptions { chains, sparsest }` plus an optional `ShapeTrace`) is, with options off, the committed `solve_s5_subspace` count for count (unit test `plain_options_keep_the_solutions_and_off_is_the_committed_solve`, and the committed arm below reproduces §11.17's counts exactly).
- **Arms.** `committed` (§11.17); `sparsest` (pivot = sparsest candidate row); `sparsest+chains` (eigen step along the chains of e_1); `sparsest+chains+replay` (F4Remake: the first solve records which rows reduce to zero, later solves skip them and top up from the held rows if the rank falls short; a shape mismatch runs the full solve, both charged).
- **Instances.** `p = 269`, seed 1: §11.17's curve, 149-point factor base and residual stream. Residual 0 is the constructed four-point sum; the residual stream advances with the committed arm's solve, as it did in §11.17, so residuals 0–3 are §11.17's four.
- **Pairing and order.** Every arm of a residual starts from a clone of the same generator state. The arm order rotates by one per residual (a Latin square over the 4-arm run; each arm first, second and third over the 3-arm run).
- **Correctness.** Every solve's decomposition set is compared with the meet-in-the-middle oracle.
- **Unit.** `F_p` multiplications, counted as §11.17 counts them (elimination, normal forms, characteristic polynomial, roots, eigenvectors, Weil restriction, splitting). Counts do not depend on contention.
- **Host.** Apple M4 Pro, 14 cores, 48 GiB, macOS, `RAYON_NUM_THREADS=4`, no isolation; load average about 26 from other sessions' jobs (`raw/host.txt`). Wall times are recorded but contended, so they are descriptive only.

Results

S_5, residuals 0–12 (`raw/s5_p269_seed1_r13.json`)

arm	n	mean C_4	$\log_2$	committed / arm, ratio of means	paired ratio, min–max	oracle-equal
committed	13	$1.2093 \cdot 10^{12}$	40.14	1	—	13/13
sparsest + chains	13	$1.6750 \cdot 10^{11}$	37.29	**7.22**	7.12–7.39	13/13
sparsest + chains + replay	12	$\mathbf{8.690 \cdot 10^{10}}$	36.34	**13.90**	7.24–15.37	13/13

The replay arm's first solve (residual 0, $1.689 \cdot 10^{11}$) records the shape and is excluded from its mean, as #1792 excluded it. Of its 12 later solves, 11 replayed (7.89–$8.04 \cdot 10^{10}$) and one, residual 3, fell back to the full solve ($1.656 \cdot 10^{11}$; the triangularised leading monomials differed, the only check before elimination, so the failed attempt cost almost nothing). The mean charges that fallback.

Phase means (replay arm): elimination $3.17 \cdot 10^{10}$, normal forms $5.20 \cdot 10^{9}$, characteristic polynomial $4.97 \cdot 10^{10}$, eigenvectors $2.9 \cdot 10^{8}$. Against the committed solver, elimination and normal forms ($5.82 \cdot 10^{11}$ and $5.46 \cdot 10^{11}$) no longer dominate: the characteristic polynomial of the chain step, $0.72 \cdot D^3$ with $D = 4{,}096$, is now 57 % of C_4.

Ablation, residuals 0–3 (`raw/s5_p269_seed1.json`, 4 arms, Latin square)

arm	n	mean C_4	committed / arm	paired min–max
committed	4	$1.2130 \cdot 10^{12}$	1	—
sparsest	4	$3.0865 \cdot 10^{11}$	**3.93**	3.78–4.11
sparsest + chains	4	$1.6758 \cdot 10^{11}$	7.24	7.21–7.26
sparsest + chains + replay	3	$1.0855 \cdot 10^{11}$	11.13	7.24–15.16

Same instances and counts as the first four residuals above (each count reproduced exactly between the two runs). Sparsest pivots alone give 3.9× here, not the 26× they gave on the D_4 system's plain five-unknown form: that elimination's fill-in was far worse.

Replay checked on nonempty decompositions (`raw/s5_p269_seed1_constructed4_replay.json`)
In the paired runs only residuals 0 and 6 decompose. To check replayed solves on decompositions that exist, four constructed residuals (planted four-point sums) were solved by the replay arm alone: the recording solve and all three replays (7.94–$7.97 \cdot 10^{10}$) return exactly the planted quadruple, equal to the oracle.

Generic systems, same solver (`raw/generic_d4-7.json`)
Four random planted equations of degree d in four unknowns, as §11.17′s scaling panel (committed counts reproduce §11.17′s: $2.30 \cdot 10^{8}$, $3.66 \cdot 10^{9}$, $3.55 \cdot 10^{10}$, $2.40 \cdot 10^{11}$). Committed / option:

d	sparsest	sparsest + chains
4	2.72	1.79
5	3.26	3.09
6	3.44	4.73
7	3.50	6.07
8 (S_5, above)	3.93	7.22

The chain step's saving grows with d; S_5 sits on the generic trend, as §11.17 found for the committed solver.

Validation (benchmarks gate, `docs/VALIDATION_GATE.md`)

The benchmarks row's `VALIDATION.json` (`benchmarks/20261019-s5-plain-c4-options/`) pins three artifacts, all in `validation/`:

- **Replay certificate** (`replay-certificate.json`, `replay.sh`). Every recorded run was re-run from a fresh release build of the committed source (`ca88b3917`) as new processes. That covers the 13-residual run, the 4-arm ablation, the constructed replay check, the generic panel and the 20 rho runs. jq, not the runner, compared each replay with the committed records: every counted phase, the decomposition and oracle sets and their agreement, and each rho run's counts and verified logarithm. **All 91 solves are identical.** Independence means a new build, new processes and a separate comparator on the same host. It was not a second machine.

- **Validator report** (`validator-report.json`, `validate.sh`). The row's headline numbers, the rho means and the band values are recomputed from the raw records with jq, which shares no code with the Rust summary that produced them. All 17 match.
- **Rho reference** (`rho_reference.json`, `curve_info.json`). It was **chosen after the measurement, to meet the gate**: the study registered no rho reference, because it measures one stage. §11.16′s thread reference is plain r-adding rho (S ≈ 1.3). This uses the stricter one:
  - **What ran:** a one-target **negation-map** rho on §11.17′s own curve, counted in the same unit (`F_p` multiplications) by the same runner and `Fp4` counters as C_4.
  - **Subgroup:** `#E(F_{269⁴}) = 5,236,052,031 = 189 · 27,703,979`. The runs are in the prime subgroup of order ℓ = `27,703,979` (`2^24.72`).
  - **Result:** 20 solves, all logarithms verified, mean $7.736 \cdot 10^5$ `F_p` multiplications. The mean S of 1.47 includes the walk's 32 precomputed multipliers, a large share at this size.
  - **Strongest symmetry:** `j(E) ∉ F_{p²}` and `a, b ≠ 0`, so no Frobenius fold applies and there are no extra automorphisms. Negation is the strongest symmetry this curve has.
  - **Same binary:** the C_4 records come from the original build. The `ca88b3917` build that ran rho reproduced every C_4 count exactly.
- **§5a band.** Rho's unpriced native work is counted from its records: per step one table lookup, one insert and one partition hash, and per walk attempt one canonicalisation and one hash. That is `5·steps + 2·look-aheads`, a mean of 25,931 operations per run. Priced at w group-addition equivalents of 97 `F_p` multiplications each, rho costs $7.74 \cdot 10^5$, $1.03 \cdot 10^6$ and $3.29 \cdot 10^6$ at w = 0, 0.1 and 1. The IC side is a lower bound: one measured decomposition, $C_4 = 8.69 \cdot 10^{10}$, without the other decompositions, the linear algebra or any unpriced work, whose counters this solver does not emit. Rho is cheaper at every point of the band, by $2.6 \cdot 10^4\times$ even at w = 1.

This row is a solver-stage improvement. The rho reference measures the distance remaining, not a comparison the row claims to win: at `p = 269` a single plain-S_5 decomposition costs about $10^5\times$ a whole rho solve.

What changed in the cost model

- `extension_measured_solve_mults`: `(4, 1.227e12)` ⟶ `(4, 8.69e10)`, i.e. $\log_2 C_4$ 40.16 ⟶ 36.34 (−3.82 bits on every `k = 4` extension IC price).
- `log2_extension_solve_mults` charged `max(measured, D³)`. D^3 is not a floor: the chain step alone costs $0.72 \cdot D^3$ here, and #1792′s D_4 oracle measured about `2^26` against the plain D^3 = `2^36` (and below its own quotient's 512^3 = `2^27`). A measured `C_k` is now charged as measured; D^3 stays the estimate where nothing is measured (`k ≠ 3, 4`), described as an estimate, not a lower bound, and labelled `degree_cubed_estimate` (was `degree_cubed_floor`) in the registry. The test asserts that a measured price below D^3 is not raised. With $8.69 \cdot 10^{10}$ = `2^36.34` the clamp would not have bound today; it would have at any further 1.27×.
- `docs/curves/jacobian-transfers.json` regenerated once (`jacobian_transfers build --budget 200000`, `check` current). Verdicts before and after: 598 `formula_advantage`, 57 `formula_marginal`, 1,917 `boundary` routes; every route class and every curve verdict identical (diffed route by route). Only the notes, the basis label and the cost-model block changed. **No boundary verdict changed**, because the registry's extension index-calculus rows are at k = 2, 3, 5, 6, 10, 11 and 14, never 4.

Derived, not measured

On §11.16′s formula $p^* = 0.095 \cdot C_4 / (1 - r_\infty)$ and its exponents, a factor 13.9 off C_4 moves the crossover n^* about $4 \log_2 13.9 \approx 15$ bits earlier: from `2^149.4` to about `2^134` at r_∞ = `0.34`. This is an extrapolation from a stage cost, as §11.17′s was.

Limits

- One prime (`p = 269`), one curve; 13 paired residuals, 11 replayed solves of the arm that sets the price. Counts vary 2.3 % across residuals with the committed solver (eigenvector step) and under 2 % within the replayed set.
- The replay fallback rate (1 of 12) is a small sample; at a fallback rate `f` the mean is about $(1 - f) \cdot 7.96 \cdot 10^{10} + f \cdot 1.66 \cdot 10^{11}$.
- Not compared with an external solver (msolve, Magma F4 + sparse FGLM) on the same system (AGENTS.md §15). The price is this repository's best verified solver, not the fastest known.
- Wall time contended (host load about 26): descriptive only. Counted multiplications are the result.

Reproduce

```
cargo run --release --example gaudry_quartic_c4_options -- --p 269 --seed 1 --residuals 13 \
    --arms committed,sparsest+chains,sparsest+chains+replay --json raw/s5_p269_seed1_r13.json
cargo run --release --example gaudry_quartic_c4_options -- --p 269 --seed 1 --residuals 4 \
    --arms committed,sparsest,sparsest+chains,sparsest+chains+replay --json raw/s5_p269_seed1.json
cargo run --release --example gaudry_quartic_c4_options -- --p 269 --seed 1 --residuals 4 --constructed 4 \
    --arms sparsest+chains+replay --json raw/s5_p269_seed1_constructed4_replay.json
cargo run --release --example gaudry_quartic_c4_options -- --generic 4,5,6,7 \
    --arms committed,sparsest,sparsest+chains --json raw/generic_d4-7.json
```

```
cargo run --release --example gaudry_quartic_c4_options_summary -- raw/s5_p269_seed1_r13.json - raw/
summary_r13.json
cargo run --release --example gaudry_quartic_c4_options_summary -- raw/s5_p269_seed1.json raw/
generic_d4-7.json raw/summary.json
MPLCONFIGDIR=$TMPDIR python3 figures/plot.py
```

`raw/generic_d4-6.json` is the first pilot of the generic panel (identical counts to the `d ≤ 6` rows of `generic_d4-7.json`). Run report: `reports/run-20261019T1851Z/`. Report PDF: `REPORT.pdf`.